

Worksheet · Section 3.12–3.13

Calling & Developing Procedures



The Big Picture

Procedures package code into reusable, named pieces. Practice tracing calls and returns, telling parameters from arguments, and writing generalized procedures — a Create Task requirement!

Part 1 · Vocab Match

Write the letter of the matching definition on each line.

- | | |
|--|---|
| _____ 1. Procedure | A. A statement that runs a procedure, like <code>area(3, 5)</code> |
| _____ 2. Parameter | B. Using a procedure knowing what it does, not how |
| _____ 3. Argument | C. The order in which statements run |
| _____ 4. Return value | D. An input variable listed in a procedure's definition |
| _____ 5. RETURN | E. A named group of instructions that can be called |
| _____ 6. Procedure call | F. Dividing a program into separate subprograms |
| _____ 7. Procedural abstraction | G. The value a procedure sends back to where it was called |
| _____ 8. Modularity | H. The actual value passed in when a procedure is called |
| _____ 9. Generalization | I. Sends back a value and ends the procedure immediately |
| _____ 10. Flow of control | J. Using parameters so one procedure works for many inputs |

Part 2 · Fill It In

Use each word from the word bank once.

argument**RETURN****modularity****abstraction****reuse****generalize****parameter****call**

1. In `PROCEDURE area(w, h)`, `w` is a _____.
2. In `area(3, 5)`, `3` is an _____.
3. _____ sends a value back and ends the procedure.
4. Writing `area(3, 5)` is a procedure _____.
5. Using a procedure without knowing how it works is procedural _____.
6. Splitting a program into separate procedures is _____.
7. Adding parameters lets you _____ a procedure.
8. Procedures let programmers _____ code instead of copying it.

Worksheet · Section 3.12–3.13

Practice

Part 3 · Trace the Calls

```
PROCEDURE f(x, y)
{
  IF (x > y) { RETURN (x - y) }
  RETURN (x + y * 2)
}
```

Call	Returns / displays
f(10, 3)	_____
f(2, 5)	_____
f(4, 4)	_____
f(f(9, 2), 1)	_____
DISPLAY(f(1, 1) + f(5, 0))	_____

Part 4 · Parameter or Argument?

```
PROCEDURE greet(name, times)
{
  REPEAT times TIMES
  { DISPLAY(name) }
}
greet("Ana", 2)
greet(user, 3)
```

Item	Parameter or argument?
name	_____
times	_____
"Ana"	_____
2	_____
user	_____

greet("Ana", 2) displays _____

Part 5 · Generalize It

A program has `drawRed()`, `drawBlue()`, and `drawGreen()`. Each draws a circle with identical code except the color. Rewrite them as ONE procedure, show three calls, and give one benefit.

Part 6 · Write a Procedure

Write `max3(a, b, c)` that RETURNS the largest of three numbers. Then give the value of `max3(4, 9, 2)`.



Think:

Parameters are Placeholders (in the definition). Arguments are Actual values (in the call). And RETURN ends the procedure right away.

Worksheet · Section 3.12–3.13

Exam Practice

Part 7 · Exam Practice

★ Warm-up

1. After a procedure finishes, where does the program continue?

- A. Inside the procedure again
- B. At the first line of the program
- C. At the statement right after the call
- D. At the end of the program

★★ On Level

2. What is displayed?

```
PROCEDURE sq(n)
{ RETURN (n * n) }
DISPLAY(sq(3) + sq(4))
```

- A. 25
- B. 49
- C. 12
- D. 7

★★ On Level

3. Which best describes procedural abstraction?

- A. Every procedure must have three parameters
- B. You can use a procedure by knowing what it does, not how it works
- C. Procedures can't call other procedures
- D. Procedures make programs run faster

★★★ Exam Hard

4. What does `addAll([4, 5, 6])` return?

```
PROCEDURE addAll(lst)
{
  total ← 0
  FOR EACH x IN lst
  {
    total ← total + x
  }
  RETURN (total)
}
```

- A. 4
- B. 0
- C. 15
- D. 9

★★★ Exam Hard

5. Which TWO are benefits of procedures with parameters? (*Select two.*)

- A. Changes to an algorithm only need to be made in one place
- B. Every program runs faster
- C. Code can be reused with different inputs
- D. Programs can never have errors

Part 8 · Written Response

Create Task connection: describe a procedure you could write for your Create Task. Give its name, its parameter(s), what it returns or does, and where its algorithm uses sequencing, selection, and iteration.
